

ZAIZEN Financial Intelligence - Product Documentation

1 ■ ■ ■ Executive Summary, Vision & Charter

Problem Statement: Small and medium enterprises (SMEs) in the OHADA zone struggle with manual, error-prone accounting processes, lack of multi-tenant security, and non-compliance with the revised SYSCOHADA standards. Manual ledger entry and payroll calculation cost businesses hundreds of hours and expose them to significant fiscal risks.

Target Audience & Market: - **Industries:** Accounting firms, manufacturing SMEs, logistics providers, and healthcare facilities. - **User Personas:** Accountants (technical entry), Financial Managers (review/audit), HR Managers (payroll), and Business Owners (dashboards). - **Geographic Regions:** 17 OHADA member states (West & Central Africa).

Success Metrics (KPIs): - **Efficiency:** Reduce monthly closing time by 60%. - **Accuracy:** Zero-tolerance for unbalanced journal entries (Auto-validation). - **Performance:** Financial reports (Balance Sheet/Income Statement) generated in < 2 seconds. - **Security:** 100% compliance with WORM (Write Once Read Many) immutable audit trails.

2 ■ ■ ■ Product & Release Overview

Feature Releases by Version: - **v1.0 (MVP):** Multi-tenant Core, Journal Entries, Basic Ledger, and User Management. - **v1.5:** Payroll Module (SYSCOHADA compliant), CRM (Leads/Customers), and Warehouse Management. - **v2.0 (Current):** Advanced Financial Reporting (Income Statement, Balance Sheet), Ebilling integration, and Security Hardening (IDOR protection). - **v2.5 (Roadmap):** AI-powered receipt scanning, automated bank reconciliation, and mobile approval app.

Product Lifecycle: GA (General Availability). **Backward Compatibility:** All REST API v2 endpoints are stable. Database schema migrations follow EF Core standard practices.

3 ■ ■ ■ User Personas & User Stories

Personas: - **Admin:** System configuration, company creation, and role assignments. - **Accountant:** Entry of journal logs, expenses, and invoices. - **Auditor:** Read-only access to all financial data and immutable audit logs. - **Manager:** Approval of expenses and payroll, high-level reporting.

User Stories: - *As an Accountant, I want to record a double-entry transaction so that the general ledger remains balanced and compliant.* - *As a Manager, I want to lock a tax period so that past data cannot be altered after filing.* - *As an HR Officer, I want to generate automated payslips so that tax and social security*

(CNPS) contributions are calculated accurately.

4■■■ Functional Requirements Specification (FRS)

Business Rules & Logic: - **Balanced Books:** A journal entry cannot be saved unless `Total Debits == Total Credits`. - **Period Locking:** Once a fiscal month is closed, no modifications are allowed (Status: Validated). - **Tax Compliance:** Payroll must strictly follow the progressive income tax (IRPP) brackets and professional allowance caps.

Edge Cases & Negative Flows: - **Duplicate NIF:** System prevents creating two companies with the same NIF/Tax ID. - **Negative Stock:** Inventory movements cannot result in negative stock unless "Allow Overdraft" is enabled per warehouse. - **Session Timeout:** JWT expiry automatically redirects users to login to prevent unauthorized access from abandoned terminals.

5■■■ Non-Functional Requirements (NFRs)

Category	Requirement
Performance	API p95 response time < 300ms; Dashboard load < 1.5s.
Security	JWT Authentication, RSA-256 signing, Multi-tenant Isolation via SQL Filters.
Availability	99.9% target uptime via containerized deployment.
Scalability	Support for 1,000+ concurrent companies with millions of journal entries.
Compliance	Revised SYSCOHADA (2017), GDPR-compliant data handling.
Localization	Multi-language (French/English), XAF/XOF currency support.

6■■■ System Architecture & Design

ZAIZEN uses a **3-Tier N-Layer Architecture**:

1. **Presentation Tier**: React.js Single Page Application (SPA).
2. **Logic Tier**: ASP.NET Core Web API (Controllers -> Services -> Domain).
3. **Data Tier**: SQL Server (Relational) + File Storage (Receipts/Uploads).

Design Patterns: - **Repository Pattern**: For data abstraction. - **Dependency Injection**: For loose coupling of services. - **Action Filters**: For cross-cutting concerns like Multi-tenant membership checks. - **WORM Service**: For immutable transaction logging.

7■■■ Technology Stack & Infrastructure

- **Backend**: C# / .NET 8.0, Entity Framework Core.
 - **Frontend**: React 18, Vite, Vanilla CSS (Premium Design).
 - **Database**: Microsoft SQL Server 2022.
 - **Infrastructure**: Docker, Nginx (Reverse Proxy).
 - **Versioning**: Semantic Versioning (SemVer 2.0).
-

8■■■ Data & API Specifications

Core Database Schema: - Companies: Master multi-tenant table. - JournalEntries & JournalLines: Double-entry accounting core. - Employees & PaySlips: Payroll engine data. - UserCompanies: Join table for RBAC/Membership mapping.

API Contract: OpenAPI (Swagger) documented at /swagger/index.html. **Environment Variables**: ConnectionStrings:DefaultConnection, Jwt:SecretKey, CompanySettings:TransportRate.

9■■■ Security & Compliance Model

- **Authentication**: JWT (JSON Web Token) with claims-based authorization.
 - **Multi-Tenancy**: The CompanyMembershipActionFilter intercepts every request to ensure the authenticated user belongs to the requested CompanyId.
 - **Audit Trails**: Every creation/modification is logged in the AuditLogs table with Timestamp, UserID, Action, and IP Address.
 - **Encryption**: Passwords hashed with BCrypt; Data-in-transit secured via TLS 1.3.
-

■ Build, Deployment & CI/CD

1. **CI Pipeline**: Github Actions -> Build .NET & React -> Run Unit Tests.

2. **Deployment:** Docker Compose deployment to Linux VPS.
 3. **Strategy:** Rolling updates with zero-downtime database migrations.
 4. **Rollback:** Automated script to revert Docker image tag and EF Core migration in case of failure.
-

1■■■1■■■ Operational & Maintenance Runbook

- **Monitoring:** Health checks at `/health`.
 - **Backups:** Daily SQL Server backups (Full) + Hourly Transactional log backups.
 - **Disaster Recovery:** RPO: 1 hour, RTO: 4 hours.
 - **Capacity Planning:** Vertical scaling of CPU/RAM when concurrent users exceed 500.
-

1■■■2■■■ User Manual / Help System

Common Workflows: 1. **Company Setup:** Create Company -> Define Chart of Accounts -> Add Users. 2. **Daily Accounting:** New Journal Entry -> Attach Receipt -> Save & Validate. 3. **Payroll Run:** Select Month -> Update Overtime -> Generate Payslips -> Export PDF.

1■■■3■■■ Support & Troubleshooting Guide

- **Error 403:** User is not a member of the requested company.
 - **Error 400 (Balance):** The debits and credits in your journal entry do not match.
 - **Diagnostic:** Check `logs/error.txt` in the API root for stack traces.
-

1■■■4■■■ Version History & Deprecations

- **v2.1.0 (Current):** Security hardening, improved reporting performance.
 - **v2.0.0:** Major UI revamp, added Ebilling module.
 - **Deprecation:** v1.0 XML-based export will be removed in v3.0 in favor of JSON/PDF.
-

1■■■5■■■ Appendices

- **Glossary:**
- **OHADA:** Organization for the Harmonization of Business Law in Africa.
- **SYSCOHADA:** The accounting system used in OHADA countries.
- **RBAC:** Role-Based Access Control.
- **WORM:** Write Once Read Many (Immutable).

- **SLA:** 99.9% Monthly Uptime Guarantee.
- **License:** EULA - ZAIZEN Proprietary.